

4160 – Distributed and Parallel Computing

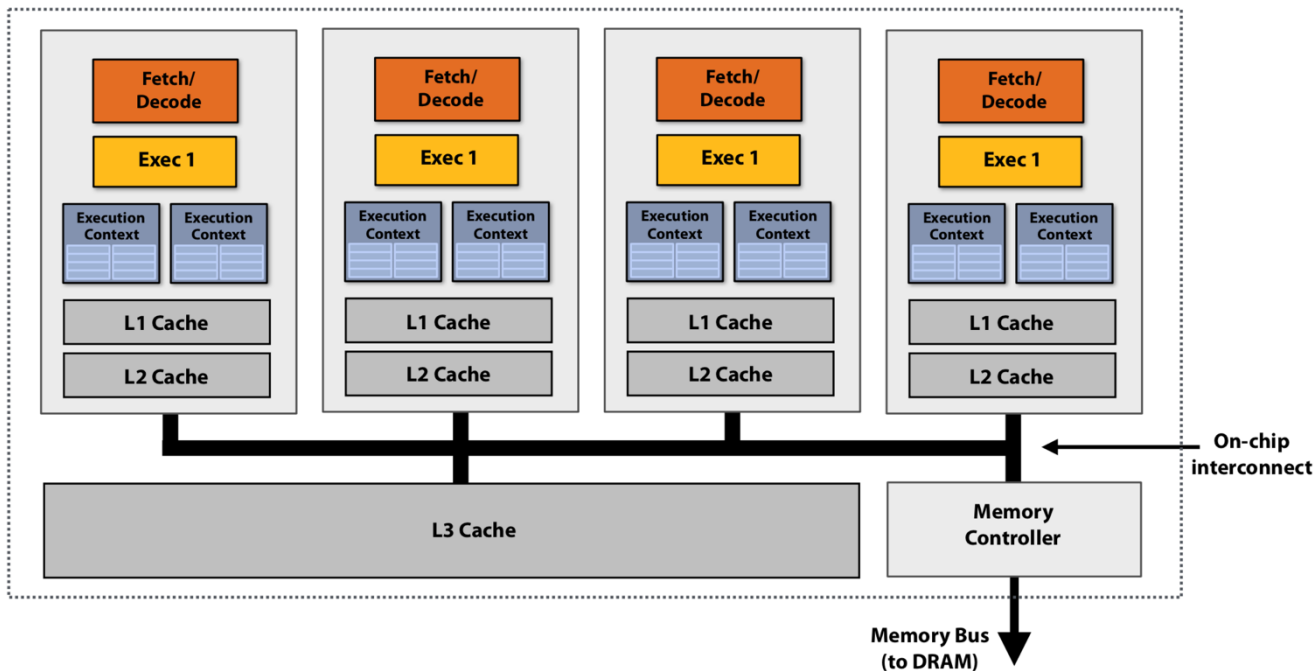
P4 – GPGPU and CUDA



CUHK

Recall: Hyperthreading

- 4 cores
- 1 core, 2 execution contexts
- A core decides which instruction from 2 execution contexts to schedule next

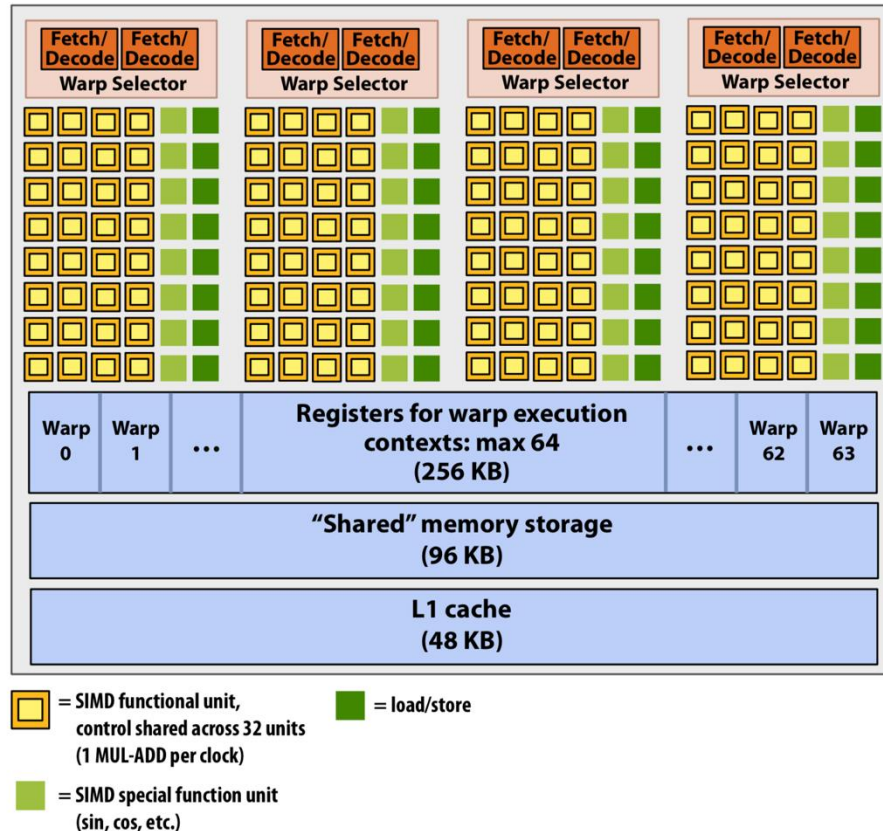


CUDA Programming

- Key concept:
 - Create as many threads as you like to parallelize your job
 - Result-oriented
- 2d-Matrix $M = A + B - C$
 - 64*64 matrix = 4096 cells
 - Kernel function { `//=thread function`

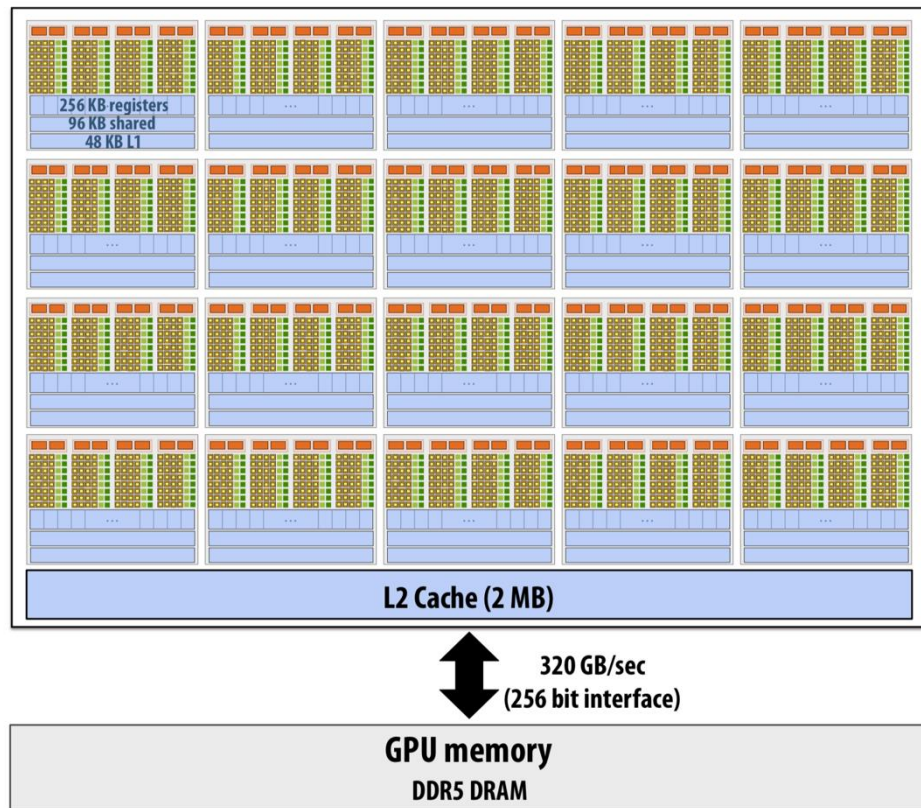
```

M[i][j] = A[i][j] + B[i][j];
M[i][j] = M[i][j] - C[i][j];
                    
```
- **Programmer** launches 4096 threads
 - Each thread computes a unique $M[i][j]$



Nvidia GTX 1080 (2016)

- 20 streaming multiprocessor
 - 20 x 4 x 32-way SIMD = 2560 cuda "cores"
- With built-in GPU "device" memory



Kernel function $\{ \neq \text{thread function} \}$

$M[i][j] = A[i][j] + B[i][j];$

$M[i][j] = M[i][j] - C[i][j];$



Warp {

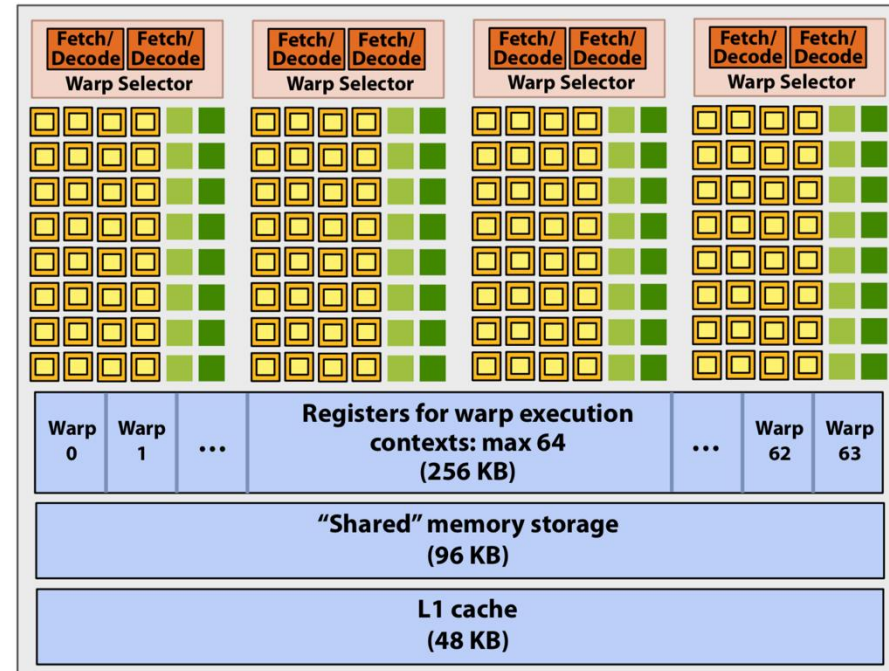
32-way SIMD+

32-way SIMD -

}

Each SM

- Collect 32 CUDA threads as a 1 **warp**
 - SIMD processing on 32 pieces of data (=32-way SIMD)
 - Consumes 32 threads in **lock-step**
- Execute 4 parallel wraps
 - Process 128 threads **simultaneously**
- Max 64 warps context
- So, each SM maximum **concurrently** works on
 - $64 \text{ warp} * 32 = 2048 \text{ CUDA threads}$



 = SIMD functional unit, control shared across 32 units (1 MUL-ADD per clock)

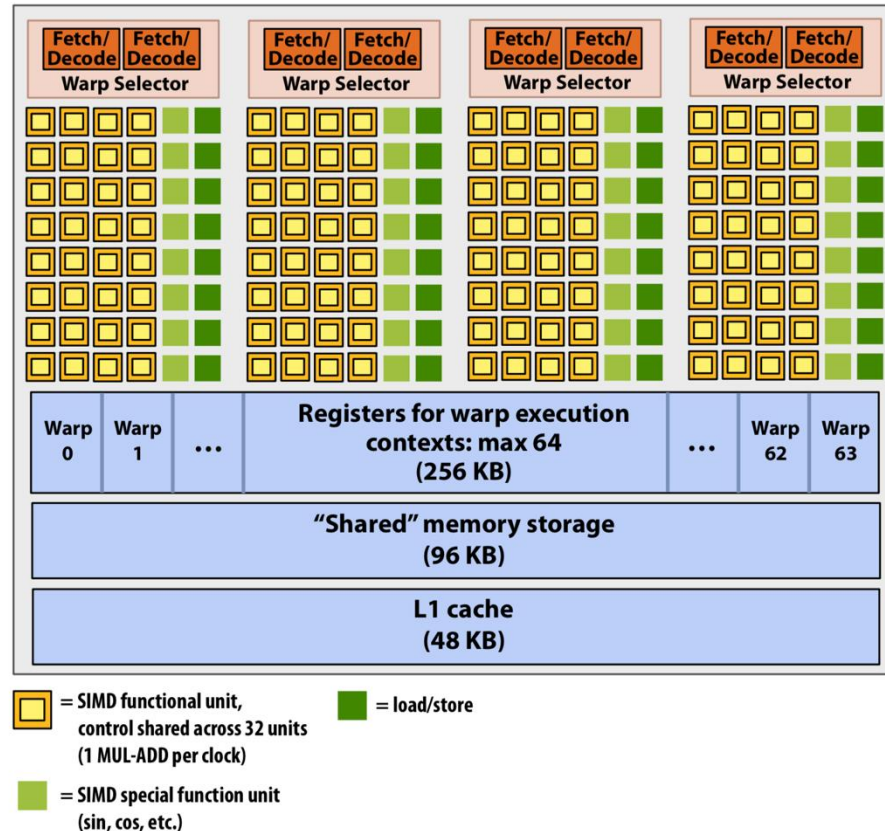
 = SIMD special function unit (sin, cos, etc.)

 = load/store



Latency hiding

- 1 SM houses 64 warps
 - Latency hiding
 - When a warp encounters a slow instruction (e.g., memory load and store)
 - Scheduler immediately picks a ready warp and context switch to it → becoming an **active** warp
 - Context switch between warps in GPU is fast
 - 1-2 cycles



Warp divergence

- Parallelism wasted and performance lost

Warp 1 {t1-t32
c[i]=a[i]+b[i] /
if (c[i]>99)

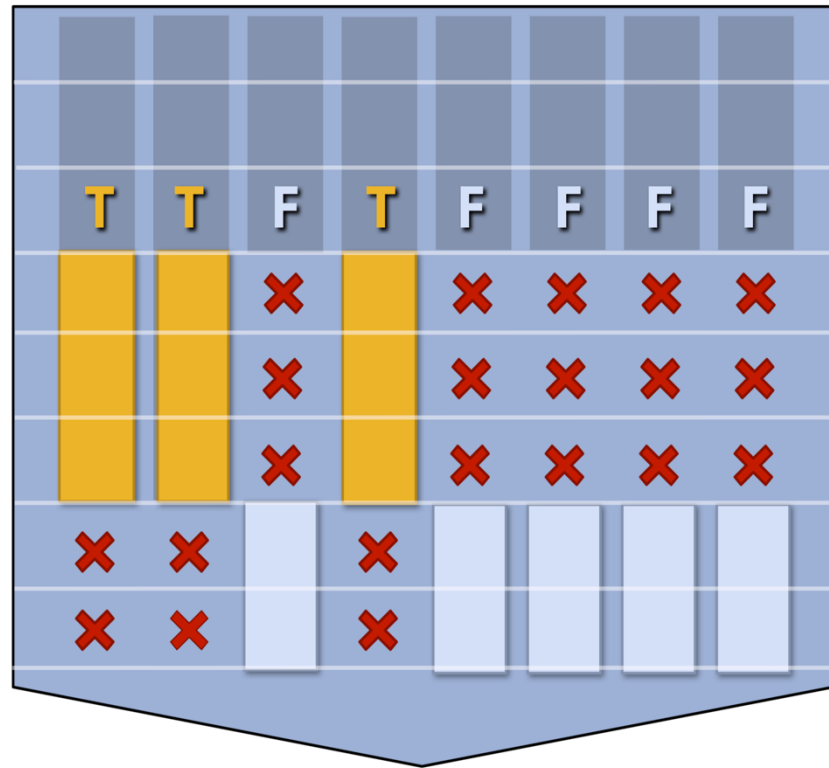
```
... //i2  
... //i3  
... //i4
```

else

```
... //i5
```

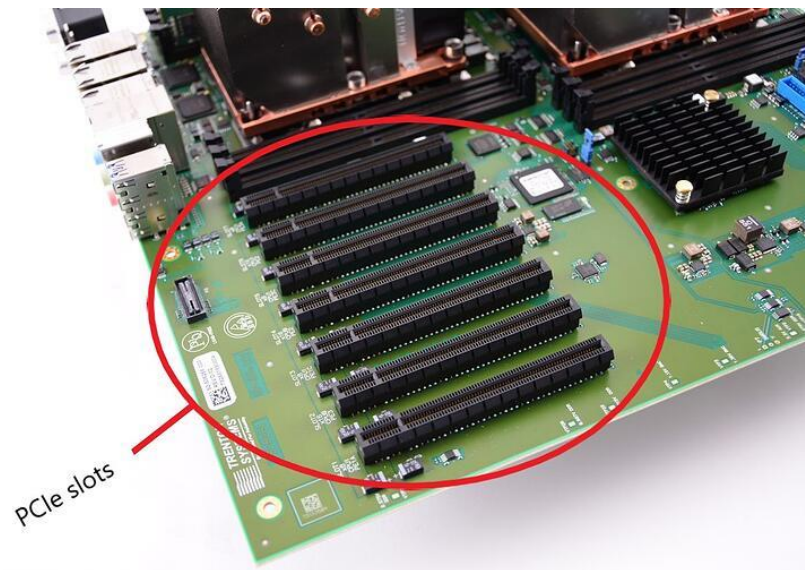
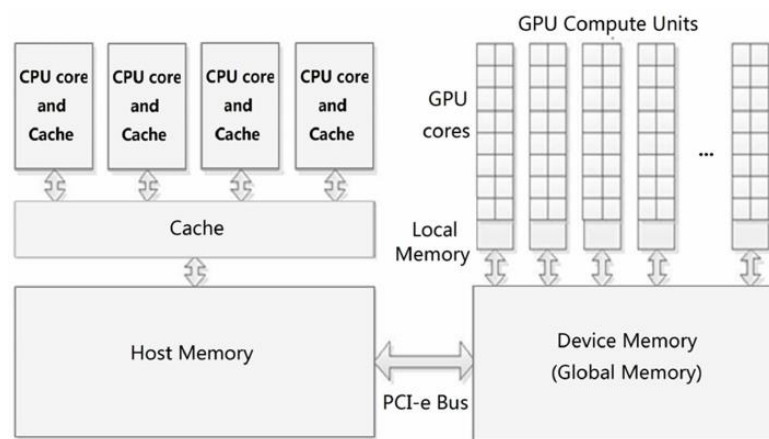
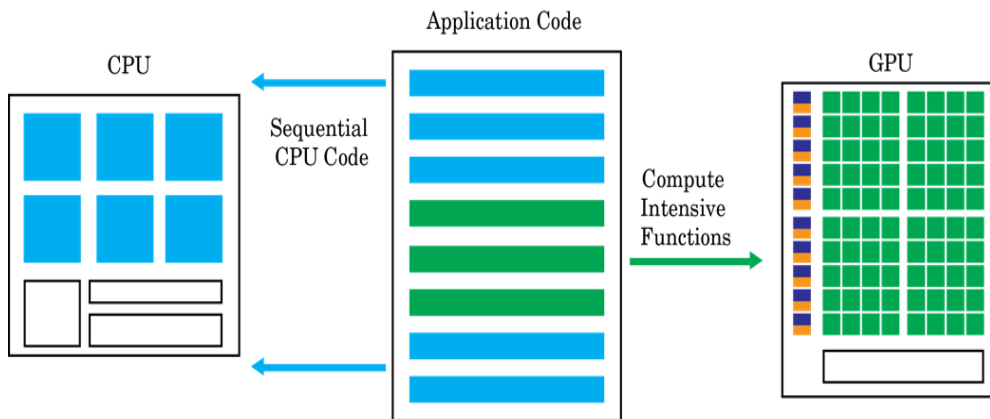
```
... //i6
```

}



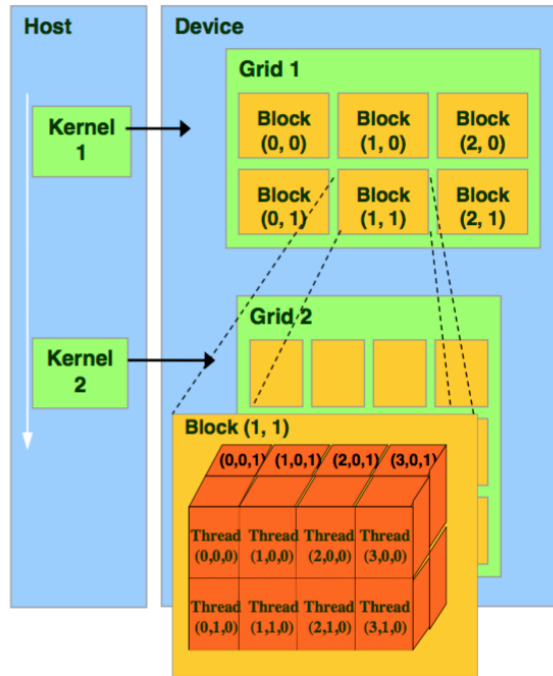
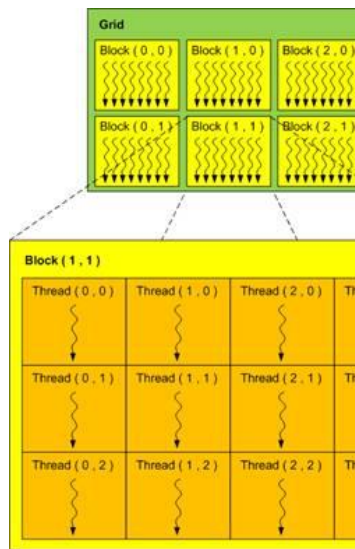
CPU and GPU as co-processor

- PCI-e 6.0: $\sim 10^2$ GB/s



CUDA Programming

- You can organize your threads into 1D/2D/3D blocks of threads
 - Within a block
 - thread is identified by threadIdx
 - max 1024 threads
- Further groups blocks into a 1D/2D/3D grids
 - Within a grid
 - Block is identified by blockIdx
 - Block size is identified by blockDim
 - Threads in the same grid execute the same kernel function



```
...  
float x = input[threadID];  
float y = func(x);  
output[threadID] = y;  
...
```

CUDA

- Thread-to-warp
 - 1D Thread block
 - 32 threads with consecutive values of threadIdx.x are combined to form 1 warp
 - Because of 32-way SIMD
 - If number of threads is not divisible by 32, then the last warp is filled with null thread
 - 2D/3D Thread block
 - Linearized in a row-wise manner and consecutive threads are combined for a warp
- Block-to-SM
 - Threads from the same block will stick to the same SM

__thread function__

each thread does it own work via the thread-id info

```
#include <iostream>
#include <math.h>
// Kernel function to add the elements of two arrays
__global__
void add(int n, float *x, float *y)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n)
        y[i] = x[i] + y[i];
}
```

$N \sim 2^{20}$ (~1 million) numbers
You decided each thread block has 256 threads
So, you need $N/\text{blocksize}$ of thread blocks

CUDA: Adding two arrays

```
int main(void)
{
    int N = 1<<20;
    float *x, *y;

    // Allocate Unified Memory – accessible from CPU or GPU
    cudaMallocManaged(&x, N*sizeof(float));
    cudaMallocManaged(&y, N*sizeof(float));

    // initialize x and y arrays on the host
    for (int i = 0; i < N; i++) {
        x[i] = 1.0f;
        y[i] = 2.0f;
    }

    // Run kernel on 1M elements on the GPU
    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;
    add<<<numBlocks, blockSize>>>(N, x, y);

    // Wait for GPU to finish before accessing on host
    cudaDeviceSynchronize();

    // Check for errors (all values should be 3.0f)
    float maxError = 0.0f;
    for (int i = 0; i < N; i++)
        maxError = fmax(maxError, fabs(y[i]-3.0f));
    std::cout << "Max error: " << maxError << std::endl;

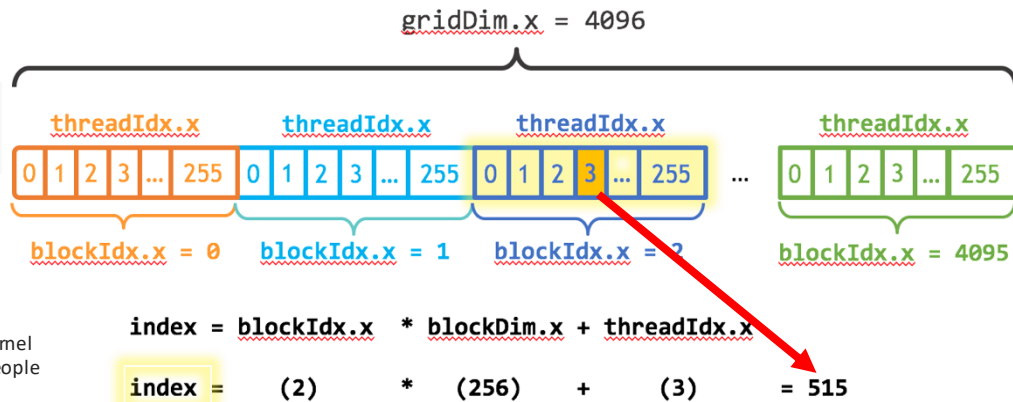
    // Free memory
    cudaFree(x);
    cudaFree(y);
}
```

CUDA Unified Memory:
a single memory address space accessible from any processor in a system
<https://devblogs.nvidia.com/unified-memory-cuda-beginners/>

CPU waits until the kernel is done before it accesses the results (because CUDA kernel launches don't block the calling CPU thread).

```
#include <iostream>
#include <math.h>
// Kernel function to add the elements of two arrays
__global__
void add(int n, float *x, float *y)
{
    int i      = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n) // it is safer in case the data array size is not a multiple of 32 as this kernel
        y[i] = x[i] + y[i]; // can be used by different programs/people
}
```



● Ex:

- 1D block with 256 threads
- 2^{20} threads / 256 = 4096 1D-blocks
- A single large grid of threads to process the entire array

```
// Run kernel on 1M elements on the GPU
int blockSize = 256;
int numBlocks = (N + blockSize - 1) / blockSize;
add<<<numBlocks, blockSize>>>(N, x, y);

// Wait for GPU to finish before accessing on host
cudaDeviceSynchronize();

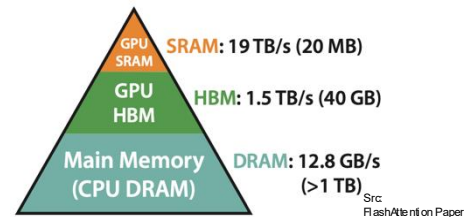
// Check for errors (all values should be 3.0f)
float maxError = 0.0f;
for (int i = 0; i < N; i++)
    maxError = fmax(maxError, fabs(y[i]-3.0f));
std::cout << "Max error: " << maxError << std::endl;

// Free memory
cudaFree(x);
cudaFree(y);
```

Efficient use of Global Device Memory

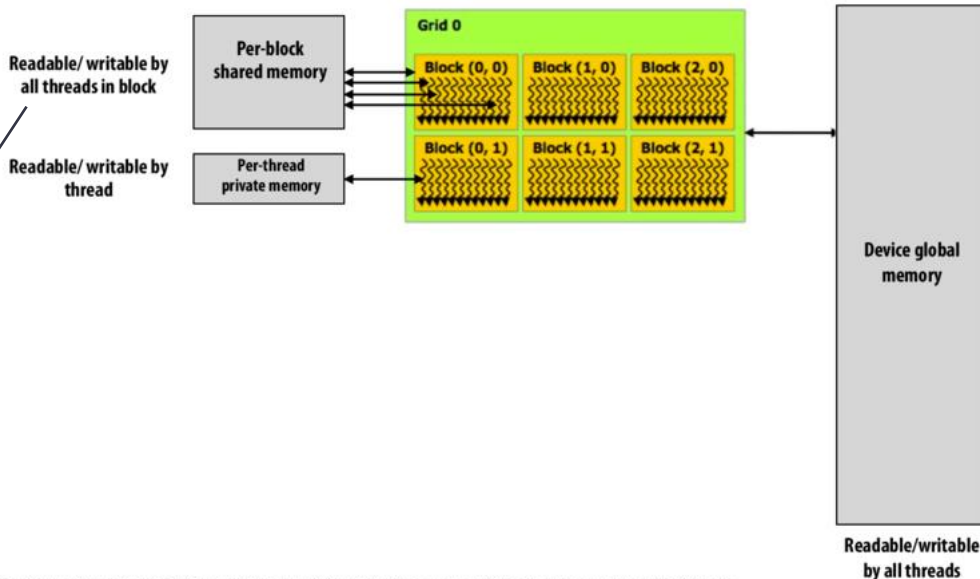
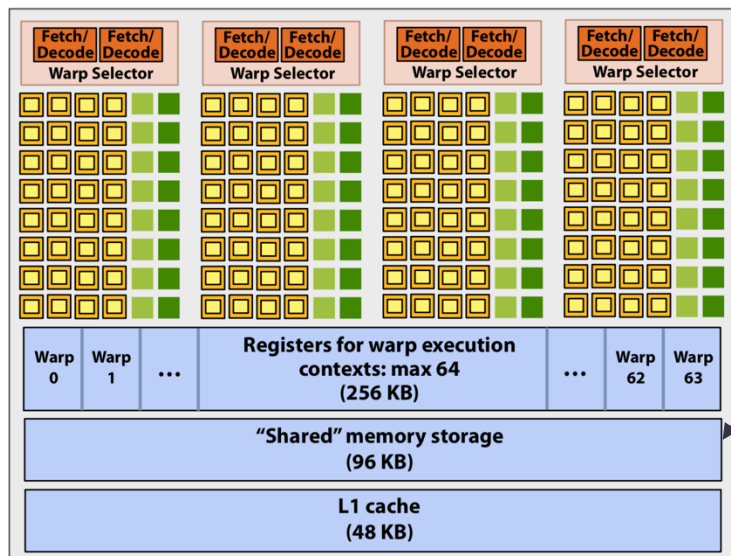
- Arrays allocated in GPU global memory are aligned to 256-byte memory segments by the CUDA driver
- CUDA *coalesces* your global memory accesses to ***memory transactions***
 - 3 sizes: 32-byte, 64-byte, or 128-byte
- A memory transaction is issued by a ***warp (32 threads)***
- The # of memory transactions strongly influences efficiency, and could be affected by:
 - How scatter the memory addresses the warp are accessing, etc.
- E.g., for a warp of 32 threads:
 - Each thread reads 1-byte? Coalesce into **one** 32-byte transaction vs. **thirty-two** 32-byte txns
 - Each thread reads 4-byte? Coalesce into **one** 128-byte transaction vs. **thirty-two** 32-byte txns

SM's Shared Memory



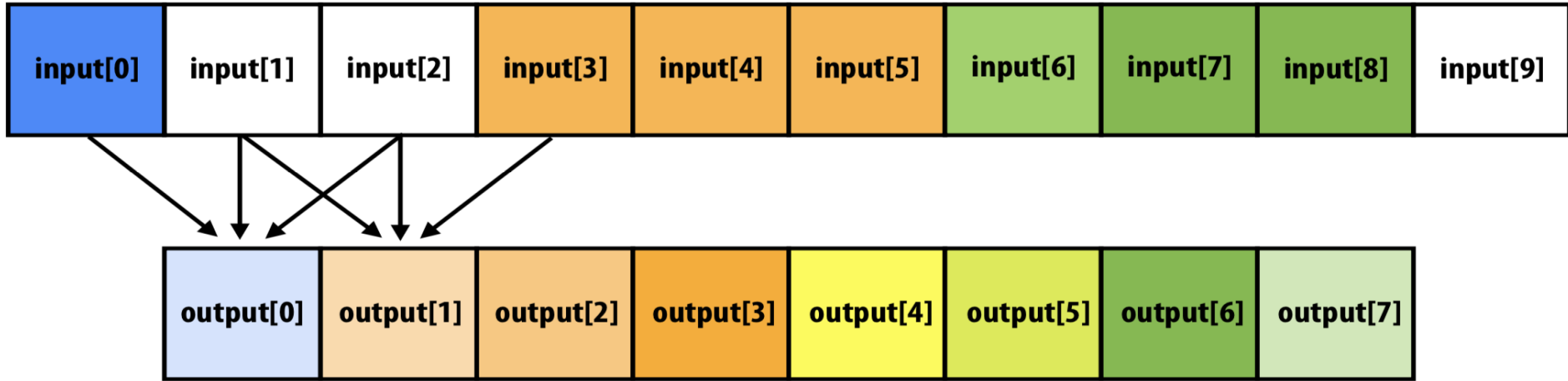
CUDA device memory model

Three** distinct types of memory visible to device-side code



** There are actually five: the "constant memory" and "texture memory" address spaces are not discussed here (carried over from graphics shading languages)

Use of shared memory – example 1D convolution



```
output[i] = (input[i] + input[i+1] + input[i+2]) / 3.f;
```



```
#define THREADS_PER_BLK 128
```

```
__global__ void convolve(int N, float* input, float* output) {  
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable  
  
    float result = 0.0f; // thread-local variable  
    for (int i=0; i<3; i++)  
        result += input[index + i];  
  
    output[index] = result / 3.f;  
}
```

each thread computes
result for one element

each thread writes result
to global memory

Vs (below): explicit load the input into _shared_ memory within an SM

```
#define THREADS_PER_BLK 128
```

```
__global__ void convolve(int N, float* input, float* output) {  
    __shared__ float support[THREADS_PER_BLK+2]; // per-block allocation  
    int index = blockIdx.x * blockDim.x + threadIdx.x; // thread local variable  
  
    support[threadIdx.x] = input[index];  
    if (threadIdx.x < 2) {  
        support[THREADS_PER_BLK + threadIdx.x] = input[index+THREADS_PER_BLK];  
    }  
  
    __syncthreads();  
  
    float result = 0.0f; // thread-local variable  
    for (int i=0; i<3; i++)  
        result += support[threadIdx.x + i];  
  
    output[index] = result / 3.f;  
}
```

+2 because 8 outputs requires 10 inputs in conv

All threads cooperatively load
block's support region from
global memory into shared
memory

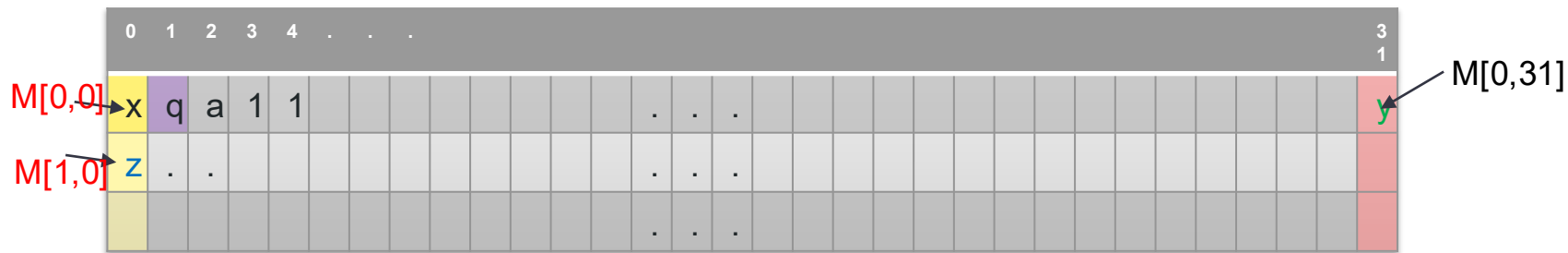
(total of 130 load instructions
instead of 3 * 128 load instructions)

barrier (all threads in block)

each thread computes
result for one element

write result to global
memory

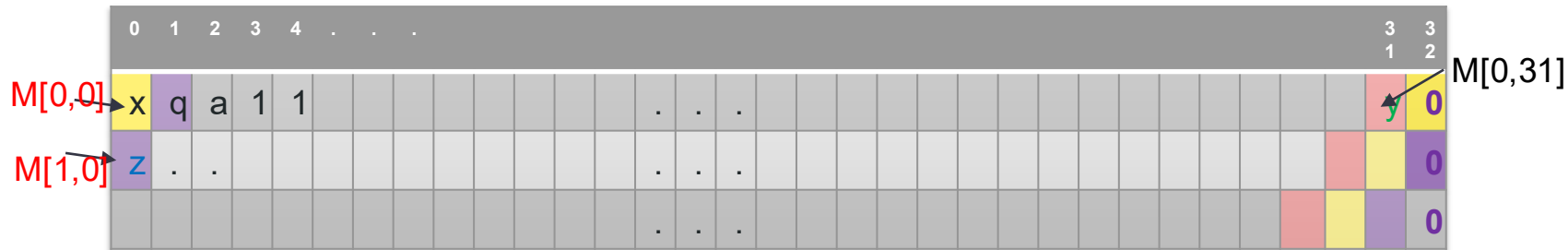
Efficient use of Shared Memory



- 96K Shared Memory is divided into 32 **banks** (bank-width: 4 bytes)
- Each bank can service (read/write) only 1 request at a time
- Concurrent access to the same bank
 - Bank Conflict! Line-up (Serial)
 - E.g., Kernel {


```
//t(0,0) accesses M[0,0], t(0,1) accesses M[0,2], ..
//t(1,0) accesses M[1,0]
//the two threads from the same wrap conflict at the same yellow bank
```

Efficient use of Shared Memory



- To resolve, pad a **0** column to M : //trade space for speed

- Initialize $M[32][33]$

- Kernel {

//t(0,0) accesses **$M[0,0]$** , ...,

//t(1,0) accesses **$M[1,0]$**

}

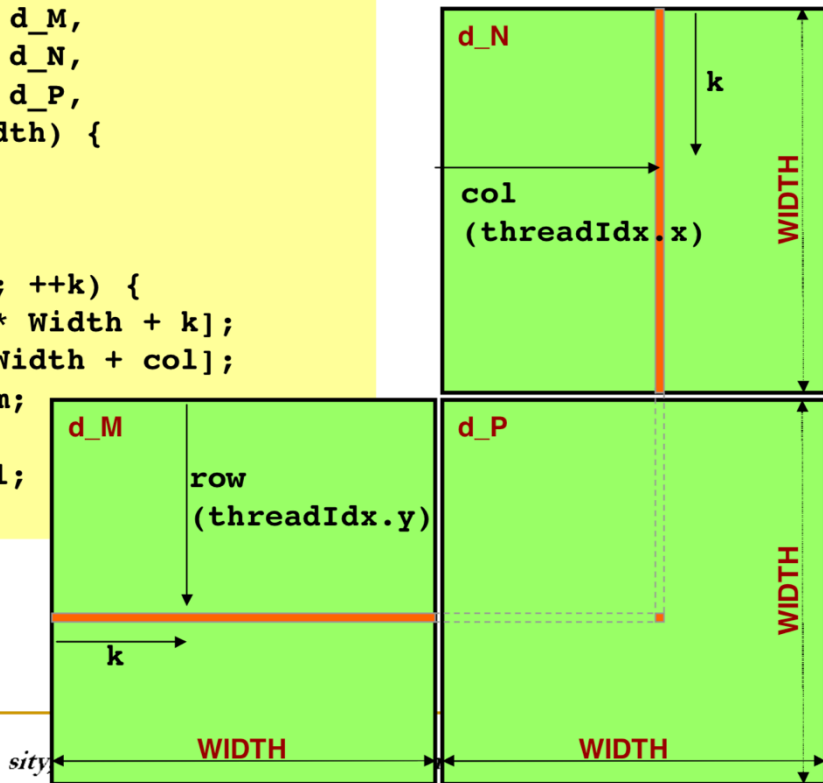
Now M **physically**:

- $M[0,0]$ and $M[0,32]$ are in the **yellow** bank
- $M[0,1]$ and $M[1,0]$ are in the **purple** bank
- Now: t(0,0) that accesses $M[0,0]$ won't conflict with t(1,0) that accesses $M[1,0]$ anymore

MM revisit

- Each thread computes 1 cell
- Limitation:
 - 1 thread block max 1024 threads
 - = at most supporting 32x32 matrix?
- CUDA often uses *ptr to access a matrix

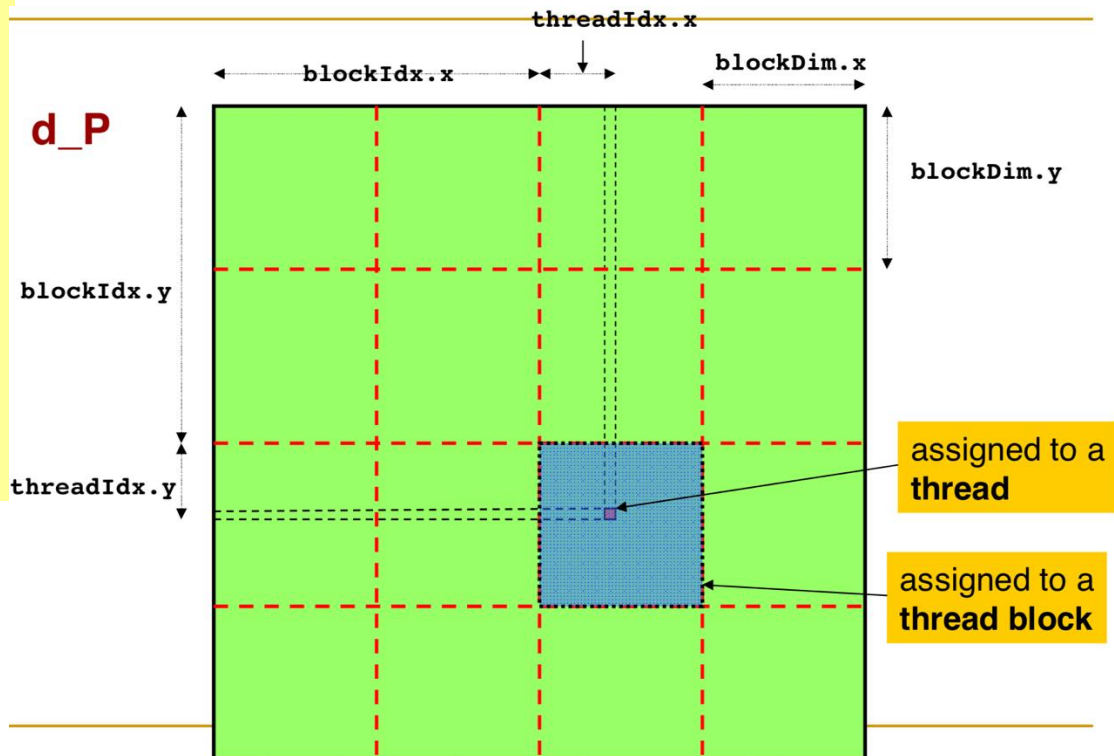
```
__global__  
void MatrixMulKernel(float* d_M,  
                    float* d_N,  
                    float* d_P,  
                    int Width) {  
    int row = threadIdx.y;  
    int col = threadIdx.x;  
    float P_val = 0;  
    for (int k = 0; k < Width; ++k) {  
        float M_elem = d_M[row * Width + k];  
        float N_elem = d_N[k * Width + col];  
        P_val += M_elem * N_elem;  
    }  
    d_p[row*Width+col] = P_val;  
}
```



Adapted From:
David Kirk/NVIDIA and Wen-mei W. Hwu, UIUC

Solution: Using a grid of thread blocks

```
__global__  
void MatrixMulKernel(float* d_M,  
                    float* d_N,  
                    float* d_P,  
                    int Width) {  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
    float P_val = 0;  
  
    for (int k = 0; k < Width; ++k) {  
        float M_elem = d_M[row * Width + k];  
        float N_elem = d_N[k * Width + col];  
        P_val += M_elem * N_elem;  
    }  
    d_p[row*Width+col] = P_val;  
}
```



Summary of CUDA

- What if the matrix is still $\gg$ # of all threads?
 - Recursive $\rightarrow$ a thread takes care of 1-cell-tile
- Use of shared memory
- Memory alignment
- Join SC team
- ...

CUDA programming = Sw/Hw co-design

 YouTube^{HK}

Search





WHY DO WE NEED A NEW LANGUAGE?

CUDA is too low level:

- Challenging to write; slows down iterative research
- Makes it hard for GPU programmers and ML experts to communicate

Existing DSLs are too high level

- Users can't define their own data-structure (e.g. sparse tensors)
- Users can't specify desired parallelization strategies



PYTORCH DEVELOPER DAY 2021
2:47 / 12:10



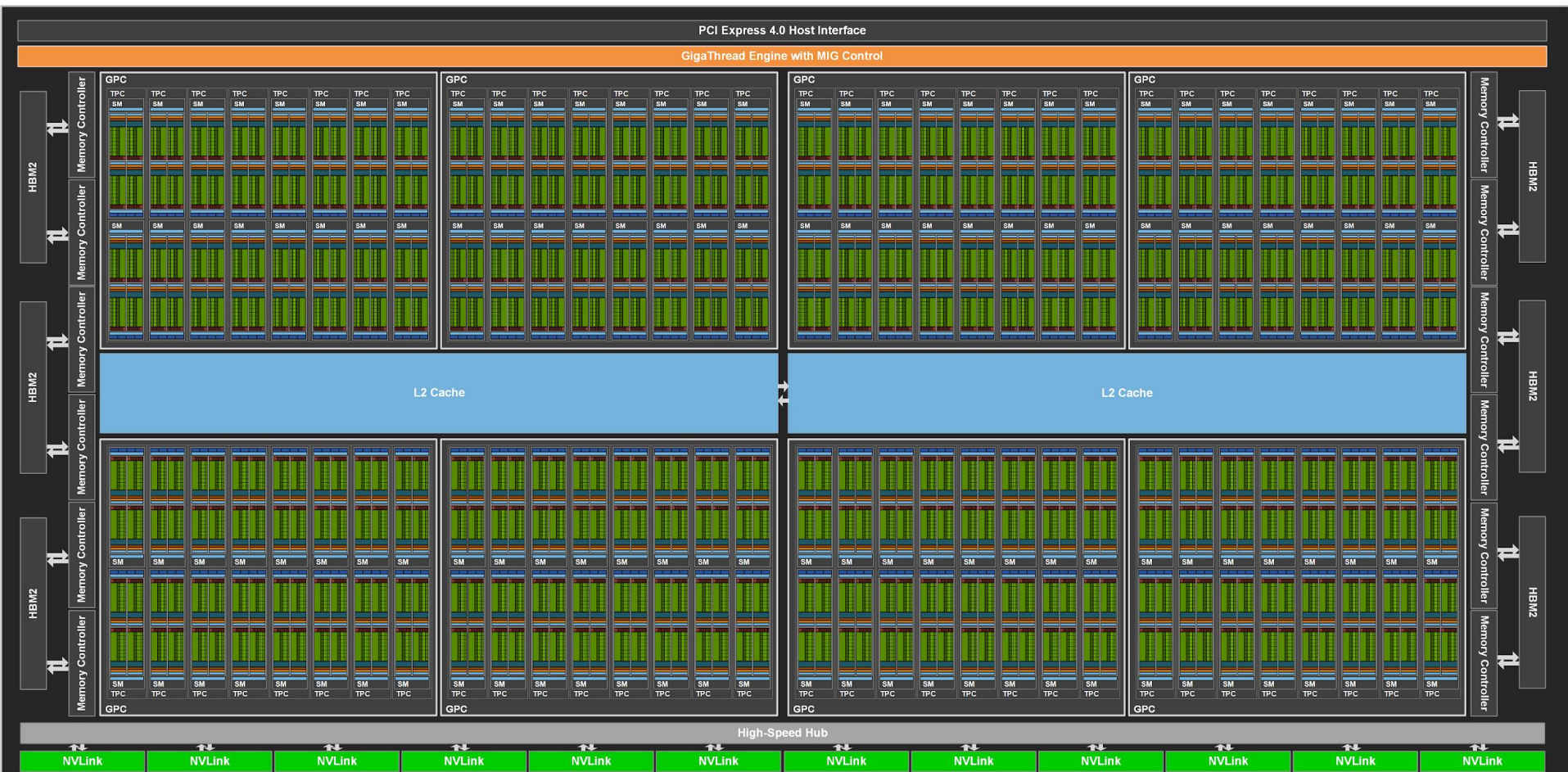




Reasoning about all these factors can be challenging, even for seasoned CUDA programmers with many years of experience. The purpose of Triton is to fully automate these optimizations, so that developers can better focus on the high-level logic of their parallel code. Triton aims to be broadly applicable, and therefore does not automatically schedule work across SMs -- leaving some important algorithmic considerations (e.g. tiling, inter-SM synchronization) to the discretion of developers.

	CUDA	TRITON
Memory Coalescing	Manual	Automatic
Shared Memory Management	Manual	Automatic
Scheduling (Within SMs)	Manual	Automatic
Scheduling (Across SMs)	Manual	Manual

Nvidia A100



A100

- 108 SM x 64-way SIMD = 6912 cuda “cores”
- + TensorCore
 - ASIC of MAC (multiply-accumulate) array for
 - $a \leftarrow a + b \times c$



- <https://www.seas.upenn.edu/~cis565/LECTURES/APSP.pdf>
- <https://devblogs.nvidia.com/even-easier-introduction-cuda/>
- <https://www.quora.com/What-is-the-difference-between-CPU-cores-and-GPU-cores-in-terms-of-computation>
- <http://users.ece.cmu.edu/~franzf/papers/pact06.pdf>
- <https://cs.nyu.edu/courses/spring12/CSCI-GA.3033-012/lecture5.pdf>
- http://icl.cs.utk.edu/classes/cosc462/2016/pdf/cosc462cuda_scheduling.pdf
- <https://medium.com/@erangadulshan.14/1d-2d-and-3d-thread-allocation-for-loops-in-cuda-e0f908537a52>
- Stanford CS149
- <https://people.maths.ox.ac.uk/gilesm/cuda/lecs/lecs.pdf>
- http://courses.cms.caltech.edu/cs179/2015_lectures/cs179_2015_lec05.pdf